

UNIT - 3

Operators and IO Statements

OPERATORS IN C

C language supports a lot of operators to be used in expressions. These operators can be categorized into the following major groups:

Arithmetic operators

Relational Operators

Equality Operators

Logical Operators

Unary Operators

Conditional Operators

Bitwise Operators

Assignment operators

Comma Operator

Sizeof Operator

ARITHMETIC OPERATORS

OPERATION	OPERATOR	SYNTAX	COMMENT	RESULT
Multiply	*	<code>a * b</code>	<code>result = a * b</code>	27
Divide	/	<code>a / b</code>	<code>result = a / b</code>	3
Addition	+	<code>a + b</code>	<code>result = a + b</code>	12
Subtraction	-	<code>a - b</code>	<code>result = a - b</code>	6
Modulus	%	<code>a % b</code>	<code>result = a % b</code>	0

Value of a=9, b=3

RELATIONAL OPERATORS

Also known as a comparison operator, it is an operator that compares two values. Expressions that contain relational operators are called *relational expressions*. Relational operators return true or false value, depending on whether the conditional relationship between the two operands holds or not.

RELATIONAL OPERATORS

OPERATOR	MEANING	EXAMPLE
<	LESS THAN	3 < 5 GIVES 1
>	GREATER THAN	7 > 9 GIVES 0
>=	GREATER THAN OR EQUAL TO	100 >= 100 GIVES 1
<=	LESS THAN OR EQUAL TO	100 <=50 GIVES 0

EQUALITY OPERATORS

C language supports two kinds of equality operators to compare their operands for strict equality or inequality. They are equal to (==) and not equal to (!=) operator.

The equality operators have lower precedence than the relational operators.

OPERATOR	MEANING
==	RETURNS 1 IF BOTH OPERANDS ARE EQUAL, 0 OTHERWISE
!=	RETURNS 1 IF OPERANDS DO NOT HAVE THE SAME VALUE, 0 OTHERWISE

LOGICAL OPERATORS

C language supports three logical operators. They are- Logical AND (&&), Logical OR (||) and Logical NOT (!).

As in case of arithmetic expressions, the logical expressions are evaluated from left to right.

A	B	A &&B
0	0	0
0	1	0
1	0	0
1	1	1

A	B	A B
0	0	0
0	1	1
1	0	1
1	1	1

A	! A
0	1
1	0

UNARY OPERATORS

Unary operators act on single operands. C language supports three unary operators. They are unary minus, increment and decrement operators.

When an operand is preceded by a minus sign, the unary operator negates its value.

The increment operator is a unary operator that increases the value of its operand by 1. Similarly, the decrement operator decreases the value of its operand by 1. For example,

```
int x = 10, y;
```

```
y = x++;
```

is equivalent to writing

```
y = x;
```

```
x = x + 1;      whereas, y = ++x;
```

is equivalent to writing

```
x = x + 1;
```

```
y = x;
```

CONDITIONAL OPERATOR

The conditional operator operator (?:) is just like an if .. else statement that can be written within expressions.

The syntax of the conditional operator is

exp1 ? exp2 : exp3

Here, exp1 is evaluated first. If it is true then exp2 is evaluated and becomes the result of the expression, otherwise exp3 is evaluated and becomes the result of the expression. For example,

large = (a > b) ? a : b

Conditional operators make the program code more compact, more readable, and safer to use as it is easier both to check and guarantee that the arguments that are used for evaluation.

Conditional operator is also known as ternary operator as it is neither a unary nor a binary operator; it takes *three* operands.

BITWISE OPERATORS

- C provides six bitwise operators for manipulating the individual bits in an integer quantity.
- Bitwise operators expect their operands to be integer quantities and treat them as bit sequence.
- Bitwise negation is a unary operator that complements the bits in its operands.

BITWISE OPERATORS

Operators	Meaning	Associativity
&	bitwiseAND	LefttoRight
 	bitwiseOR	LefttoRight
~	1'sComplement	LefttoRight
^	bitwiseExclusiveOR	LefttoRight
<<	shiftleft	LefttoRight
>>	shiftright	LefttoRight

BITWISE OPERATORS

example : Consider following set of expressions:

int a,b,c;

a = 12;

b = 9;

c = a & b; // Out put is : 8

c = a | b; // out put is : 13

The bitwise AND operation will be performed at the bit level as follows

BITWISE OPERATORS

Expression	Bits
a=12	001100
b=9	001001
c=a&b	001000
c=a b	001101

ASSIGNMENT OPERATORS

The assignment operator is responsible for assigning values to the variables. While the equal sign (=) is the fundamental assignment operator, C also supports other assignment operators that provide shorthand ways to represent common variable assignments. They are shown in the table.

ASSIGNMENT OPERATORS

OPERATOR	SYNTAX	EQUIVALENT TO
/=	variable /= expression	variable = variable / expression
%=	variable %= expression	variable = variable %expression
*=	variable *= expression	variable = variable * expression
+=	variable += expression	variable = variable + expression
-=	variable -= expression	variable = variable - expression

COMMA OPERATOR

The comma operator in C takes two operands. It works by evaluating the first and discarding its value, and then evaluates the second and returns the value as the result of the expression.

Comma separated operands when chained together are evaluated in left-to-right sequence with the right-most value yielding the result of the expression.

Among all the operators, the comma operator has the lowest precedence. For example,

```
int a=2, b=3, x=0;
```

```
x = (++a, b+=a);
```

Now, the value of x = 6.

SIZEOF OPERATOR

- size of is a unary operator used to calculate the sizes of data types.
- It can be applied to all data types.
- The operator returns the size of the variable, data type or expression in bytes.
- 'sizeof' operator is used to determine the amount of memory space that the variable/expression/data type will take. For example,
- sizeof(char) returns 1, that is the size of a character data type. If we have,
 int a = 10;
 unsigned int result;
 result = sizeof(a);
 then result = 2,

Associativity of operators

If two operators of same precedence (priority) is present in an expression, Associativity of operators indicate the order in which they execute.

Associativity of operators

PRECEDENCE AND ASSOCIATIVITY TABLE

CATEGORY	OPERATORS	ASSOCIATIVITY
Parenthesis/brackets	() [] -> . ++ --	Left to right
Unary	! ~ ++ -- (type) sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Bitwise Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right

Associativity of operators

CATEGORY	OPERATORS	ASSOCIATIVITY
Bitwise OR		Left to right
Logical AND	&&	Left to right
Logical OR		Left to right
Conditional	? :	Right to left
Assignment	= += -= *= /= %= &= ^= = <<= >>=	Right to left
Comma	,	Left to right

TYPE CONVERSION AND TYPE CASTING

Type conversion and type casting of variables refers to changing a variable of one data type into another.

While type conversion is done implicitly, casting has to be done explicitly by the programmer. We will discuss both of them here.

Type conversion is done when the expression has variables of different data types. So to evaluate the expression, the data type is promoted from lower to higher level where the hierarchy of data types can be given as: double, float, long, int, char.

TYPE CONVERSION AND TYPE CASTING

For example, type conversion is automatically done when we assign an integer value to a floating point variable. For ex,

```
float x;
```

```
int y = 3;
```

```
x = y;
```

```
Now, x = 3.0,
```

TYPE CONVERSION AND TYPE CASTING

Type casting is also known as forced conversion. It is done when the value of a higher data type has to be converted into the value of a lower data type. For example, we need to explicitly type cast an integer variable into a floating point variable.

```
float salary = 10000.00;
```

```
int sal;
```

```
sal = (int) salary;
```

Typecasting can be done by placing the destination data type in parentheses followed by the variable name that has to be converted.

FORMATTING INPUT/OUTPUT

C language supports two formatting functions

- 1. printf()**
- 2. scanf()**

printf()

The printf function is used to display information required to the user and also prints the values of the variables. Its syntax can be given as

printf (“control string”, variable list);

The parameter control string is a C string that contains the text that has to be written on to the standard output device. The prototype of the control string can be given as below

%[flags][width][.precision][length]specifier

printf()

specifier	Qualifying Input
c	For single character
d	For decimal values
f	For floating point numbers
E, e	Floating point numbers in exponential format
o	For Octal number.
s	For a sequence of (string of) characters
u	For Unsigned decimal value
x	For Hexadecimal value.

printf()

<i>flag</i>	description
-	Left-justify within the data given field width
+	Displays the data with its numeric sign (either + or -) <code>printf("%+d\n", 10);</code>
0	The number is left-padded with zero (0) instead of spaces

printf()

length	Description
h	When the argument is a short int or unsigned short int. short int i = 3; printf("%hd", i);
l	When the argument is a long int or unsigned long int for integer specifiers. long int i=200000; printf(“%ld”,i);
L	When the argument is a long double (used for floating point specifiers) long double i=200000; printf(“%Lf”,i);

printf()

	Integer	Real	Character
Format Specification w=width p=precision	%w d	%w.p f	%w c
Example	printf(“%7d”,12345);	printf(“%10.2f”,12345);	printf(“%5c”,’A’);
Output _ Consider as Space	_ _12345	_ _12345.00	_ _ _ _ _A
Description	Here there are 5 digits and %7d leaves two blank space before the 5 digit number	Here 10.2f specifies 7 digits + decimal + 2 values in precision after decimal point	Here 5c specifies the position of the character to be printed

printf()

Output:

```
#include<stdio.h>
#include<conio.h>
void main()
{
    float num=12.346;
    clrscr();
    printf("\n %5.2f",num);
    printf("\n %-10.2f",num);
    printf("\n %07.2f",num);
    getch();
}
```

12.35

12.35_ _ _ _ _

0012.35

scanf()

The scanf() is used to read formatted data from the keyboard. The syntax of the scanf() can be given as,

scanf("control string", &arg1, &arg2,&argn);

The control string specifies the type and format of the data that has to be obtained from the keyboard and stored in the memory locations pointed by the arguments arg1, arg2,..., argn.

scanf()

The prototype of the control string can be give as:

[%[width][modifiers]type]

width is an optional argument that specifies the maximum number of characters to be read.

scanf()

Modifiers is an optional argument that can be h, l or L for the data pointed by the corresponding additional arguments. Modifier h is used for short int or unsigned short int, l is used for long int, unsigned long int or double values. Finally, L is used long double data values.

Type is same as specifier in printf()

scanf()

specifier	Qualifying Input
c	For single character
d	For decimal values
f	For floating point numbers
E, e	Floating point numbers in exponential format
o	For Octal number.
s	For a sequence of (string of) characters
u	For Unsigned decimal value
x	For Hexadecimal value.

scanf()

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int i;
    clrscr();
    printf("\n Enter Value of i :");
    scanf("%d",&i);
    printf("\n The Entered Value of i is : %d",i);
    getch();
}
```